

TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 2: Hồi quy logistic

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1	Mục tiêu bài thực hành	2
1.1	Môi trường làm việc	2
2	Bộ dữ liệu sinh viên	2
2.1	Bài toán đặt ra	3
2.2	Ba cột trong tệp dữ liệu	3
2.3	Đọc dữ liệu	3
2.4	Vì sao không dùng lại đường thẳng của bài 1	5
3	Hàm sigmoid	6
3.1	Công thức và ba tính chất	6
3.2	Tự cài đặt và kiểm chứng	6
4	Khớp mô hình bằng scikit-learn	8
4.1	Ba bước quen thuộc	8
4.2	Đọc kết quả	10
5	Chăm điểm mô hình	10
5.1	Ma trận nhầm lẫn	11
5.2	Chia dữ liệu rời chăm điểm	11
5.3	Bốn thước đo	13
6	Ngưỡng quyết định	14
6.1	Mô hình trả về xác suất, không phải nhãn	14
6.2	Tự áp ngưỡng	15
6.3	Chọn ngưỡng theo bài toán	16
7	Thêm biến đầu vào thứ hai	17
7.1	Trong mã chỉ đổi đúng một chỗ	17
7.2	Đọc kết quả	18
8	Bài tập	19
9	Phụ lục A, bảng tra nhanh	21
10	Phụ lục B, lỗi thường gặp	22

1. Mục tiêu bài thực hành

Buổi trước sinh viên đã dạy máy đoán một con số, cụ thể là giá một căn hộ. Buổi này máy sẽ đoán một nhãn, cụ thể là một sinh viên qua môn hay rớt môn. Đây là bài toán phân loại đầu tiên trong cả môn học.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. Giải thích được vì sao đường thẳng của bài 1 không dùng được cho nhãn 0 và 1.
2. Tự cài đặt hàm sigmoid và giải thích được vì sao nó luôn cho kết quả trong khoảng 0 tới 1.
3. Dùng `LogisticRegression` của `scikit-learn` để khớp mô hình trên dữ liệu thật.
4. Đọc được xác suất mà mô hình trả về và biết cách biến nó thành nhãn.
5. Lập được ma trận nhầm lẫn và tính được `accuracy`, `precision`, `recall`, F_1 .
6. Tự đổi ngưỡng quyết định và giải thích được vì sao `precision` với `recall` đi ngược chiều nhau.
7. Mở rộng mô hình sang nhiều biến đầu vào chỉ bằng một sửa đổi nhỏ trong mã.

Cách học bài này

Sinh viên gõ lại từng đoạn mã rồi chạy, sau đó so kết quả trên màn hình với kết quả in trong tài liệu. Nếu lệch thì đọc kỹ thông báo lỗi trước khi hỏi. Dòng cuối của thông báo lỗi gần như luôn nói rõ tệp nào và dòng nào đang sai.

1.1. Môi trường làm việc

Bài này dùng đúng môi trường đã dựng ở bài 1, không phải cài thêm gì. Sinh viên chỉ cần mở cửa sổ lệnh, bật lại môi trường ảo, rồi vào thư mục gốc của bài thực hành.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai02_hoi_quy_logistic
```

Nhắc lại lỗi hay gặp nhất

Mọi lệnh `python` trong tài liệu này đều chạy từ thư mục gốc của bài, không chạy từ bên trong thư mục `code`. Trong mã, đường dẫn tới dữ liệu được viết là `data/sinh_vien.csv`, và Python hiểu đường dẫn đó so với thư mục sinh viên đang đứng. Đứng sai chỗ thì Python báo `FileNotFoundError`.

Bảng 1: Sáu tệp mã của buổi thực hành, chạy theo đúng thứ tự này.

Mục	Tệp	Nội dung
2	<code>c1_doc_du_lieu.py</code>	Đọc bộ dữ liệu và xem qua một lượt
3	<code>c2_sigmoid.py</code>	Tự cài đặt hàm sigmoid
4	<code>c3_khop_mo_hinh.py</code>	Khớp mô hình bằng <code>scikit-learn</code>
5	<code>c4_danh_gia.py</code>	Ma trận nhầm lẫn và bốn thước đo
6	<code>c5_doi_nguong.py</code>	Đổi ngưỡng quyết định
7	<code>c6_hai_bien.py</code>	Thêm biến đầu vào thứ hai

2. Bộ dữ liệu sinh viên

2.1. Bài toán đặt ra

Một giảng viên muốn biết trước sinh viên nào đang có nguy cơ rớt môn, để còn kịp nhắc nhở trước kỳ thi. Trong tay thầy có kết quả của 120 sinh viên khóa trước, kèm theo số giờ mỗi bạn đã ôn tập. Câu hỏi là từ số giờ ôn tập, ta đoán được khả năng qua môn tới mức nào.

Điểm khác với bài 1 nằm ở thứ cần đoán. Ở bài 1, thứ cần đoán là giá nhà, một con số trượt liên tục. Ở bài này, thứ cần đoán chỉ có hai khả năng, qua hoặc rớt. Bài toán chỉ có hai nhãn như vậy gọi là phân loại nhị phân (*binary classification*).

Quy ước nhãn phải nhớ suốt bài

Ta viết nhãn thành số cho máy tính được. Số 1 nghĩa là **qua môn**, số 0 nghĩa là **rớt môn**. Quy ước này giữ nguyên trong toàn bộ tài liệu, và mọi công thức phía sau đều dựa vào nó.

2.2. Ba cột trong tệp dữ liệu

Tệp dữ liệu là `data/sinh_vien.csv`, gồm 120 dòng và 3 cột.

Bảng 2: Ba cột của tệp dữ liệu, kèm đơn vị và khoảng giá trị.

Tên cột	Ý nghĩa	Đơn vị	Khoảng giá trị
<code>gio_on</code>	Số giờ ôn tập trước kỳ thi	giờ	0.0 tới 29.5
<code>diem_giua_ky</code>	Điểm bài kiểm tra giữa kỳ	điểm	0.0 tới 10.0
<code>qua_mon</code>	Kết quả cuối môn	0 hoặc 1	0 là rớt, 1 là qua

Cột `qua_mon` là thứ ta muốn đoán, gọi là biến mục tiêu (*target*). Hai cột còn lại là căn cứ để đoán, gọi là đặc trưng (*feature*). Từ mục 3 tới mục 6 ta chỉ dùng một đặc trưng là `gio_on` cho dễ hình dung. Tới mục 7 mới thêm `diem_giua_ky` vào.

Về bộ số liệu

Đây là số liệu mô phỏng dùng cho mục đích giảng dạy, không phải số liệu khảo sát của một lớp cụ thể nào. Sinh viên đừng dùng các con số trong bài để kết luận về việc học thật của mình.

2.3. Đọc dữ liệu

```

1 # -*- coding: utf-8 -*-
2 """Bước 1: đọc bộ dữ liệu sinh viên và xem qua một lượt.
3
4 Chú thích trong tệp viết tiếng Việt có dấu, nhưng phần in ra màn hình cố ý
5 viết không dấu. Cửa sổ lệnh Windows thường không hiện được chữ có dấu.
6 """
7
8 import pandas as pd
9
10 df = pd.read_csv("data/sinh_vien.csv")
11
12 print("Kích thước bảng (số dòng, số cột):", df.shape)
13 print()
14
```

```

15 print("Nam dong dau tien:")
16 print(df.head())
17 print()
18
19 # Cột qua_mon chỉ có hai giá trị, đếm số lần xuất hiện từng giá trị
20 print("So sinh vien theo ket qua:")
21 print(df["qua_mon"].value_counts())
22 print()
23
24 print("Ty le qua mon:", round(df["qua_mon"].mean(), 4))
25 print()
26
27 # So sánh số giờ ôn trung bình của hai nhóm
28 print("So gio on trung binh theo nhom:")
29 print(df.groupby("qua_mon")["gio_on"].mean().round(2))

```

Ba lệnh mới so với bài 1, sinh viên cần nhớ.

- `df["qua_mon"].value_counts()` đếm xem mỗi giá trị xuất hiện bao nhiêu lần. Với cột chỉ có 0 và 1, lệnh này cho biết ngay lớp nào nhiều hơn.
- `df["qua_mon"].mean()` lấy trung bình của cột 0 và 1, mà trung bình của một cột toàn 0 với 1 chính là tỷ lệ số 1. Đây là mẹo rất hay dùng trong phân loại nhị phân.
- `df.groupby("qua_mon")["gio_on"].mean()` tách dữ liệu thành hai nhóm theo nhãn rồi tính trung bình số giờ ôn của từng nhóm.

Chạy tệp này bằng lệnh sau.

```
python code/c1_doc_du_lieu.py
```

```
Kich thuc bang (so dong, so cot): (120, 3)
```

```
Nam dong dau tien:
```

	gio_on	diem_giua_ky	qua_mon
0	5.5	3.0	0
1	19.0	6.0	1
2	14.0	7.7	0
3	11.0	3.1	0
4	10.5	5.8	1

```
So sinh vien theo ket qua:
```

```
qua_mon
1    71
0    49
Name: count, dtype: int64
```

```
Ty le qua mon: 0.5917
```

```
So gio on trung binh theo nhom:
```

```
qua_mon
0    8.29
```

```
1    19.89
Name: gio_on, dtype: float64
```

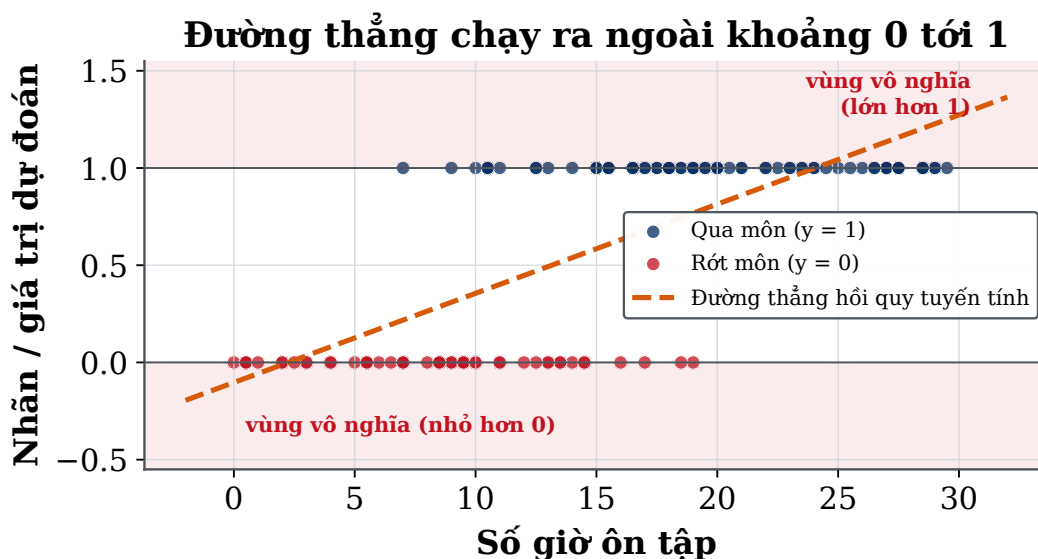
Có hai điều đáng đọc kỹ trong kết quả.

Thứ nhất, hai lớp không lệch nhau quá đáng, 71 bạn qua và 49 bạn rớt, tỷ lệ qua là 0.5917. Đây là chuyện tốt, vì khi hai lớp lệch nhau nhiều thì việc đánh giá mô hình khó hơn hẳn. Mục 5 sẽ nói kỹ về chuyện đó.

Thứ hai, và đây mới là con số quan trọng nhất, nhóm rớt môn ôn trung bình 8.29 giờ còn nhóm qua môn ôn trung bình 19.89 giờ. Hai con số cách nhau hơn hai lần, nghĩa là số giờ ôn thật sự phân biệt được hai nhóm. Nếu hai con số này gần bằng nhau thì có cố mấy cũng không xây được mô hình tốt, và ta phải đi tìm đặc trưng khác.

Thói quen nên giữ

Với mọi bài phân loại, việc đầu tiên nên làm là so trung bình của từng đặc trưng giữa các lớp. Đặc trưng nào có trung bình chênh lệch rõ thì đặc trưng đó hữu ích. Đây là cách kiểm tra rẻ tiền nhất trước khi bỏ công huấn luyện.



Hình 1: Nếu ép dùng đường thẳng của bài 1 cho nhãn 0 và 1, đường thẳng chạy ra ngoài khoảng cho phép ở cả hai đầu.

2.4. Vì sao không dùng lại đường thẳng của bài 1

Hình 1 cho thấy chuyện gì xảy ra nếu ta coi nhãn 0 và 1 là hai con số rồi khớp một đường thẳng. Trên chính dải dữ liệu, đường thẳng nhận giá trị từ -0.103 tới 1.250 . Có 25 trong 120 bạn bị gán một giá trị nằm ngoài khoảng 0 tới 1.

Một giá trị như 1.250 không đọc được thành gì cả. Khả năng qua môn bằng 125 phần trăm là câu vô nghĩa. Gốc rễ của vấn đề là một đường thẳng kéo dài ra hai phía thì vô tận, nên không có cách nào giữ nó trong khoảng 0 tới 1.

Điều ta cần là một hàm nhận vào một số thực bất kỳ rồi trả ra một số luôn nằm giữa 0 và 1. Hàm đó tên là sigmoid, và mục tiếp theo dành trọn cho nó.

3. Hàm sigmoid

3.1. Công thức và ba tính chất

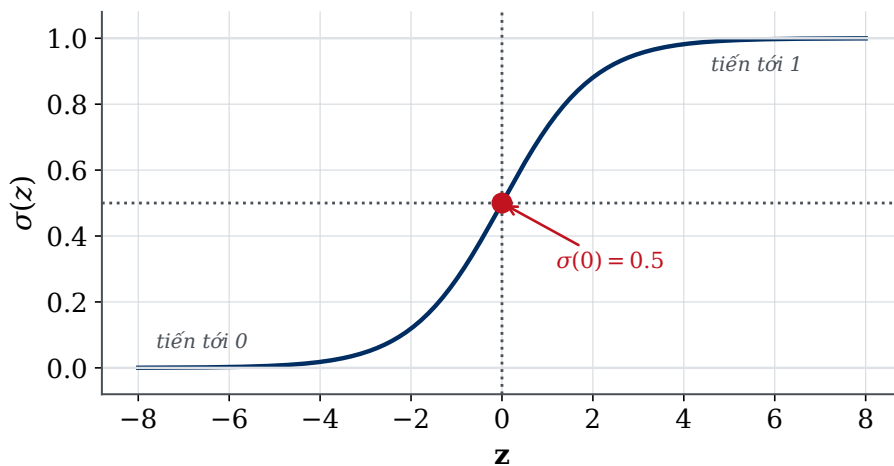
Hàm sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Chữ σ đọc là xích ma, còn e là hằng số Euler, xấp xỉ 2.718.

Sinh viên chưa cần chứng minh gì cả, chỉ cần nắm ba tính chất sau.

- Khi z rất âm thì e^{-z} rất lớn, nên mẫu số rất lớn và $\sigma(z)$ tiến về 0.
- Khi z rất dương thì e^{-z} tiến về 0, nên mẫu số tiến về 1 và $\sigma(z)$ tiến về 1.
- Khi z bằng 0 thì e^0 bằng 1, nên $\sigma(0)$ đúng bằng $1/2$.



Hình 2: Đồ thị hàm sigmoid. Đường cong có hình chữ S, ép mọi số thực về khoảng 0 tới 1 và đi qua điểm 0.5 tại gốc.

Tên gọi sigmoid nghĩa là có hình chữ S, nhìn hình 2 là thấy ngay vì sao.

3.2. Tự cài đặt và kiểm chứng

```
1 # -*- coding: utf-8 -*-
2 """Bước 2: làm quen hàm sigmoid bằng tay trước khi gọi thư viện."""
3
4 import numpy as np
5
6
7 def sigmoid(z):
8     """Ep mot so thuc bat ky ve khoang tu 0 toi 1."""
9     return 1 / (1 + np.exp(-z))
10
11
12 print("Bang gia tri cua ham sigmoid")
13 print(" z          sigmoid(z)")
14 for z in [-6, -4, -2, -1, 0, 1, 2, 4, 6]:
```

```

15     print(f"{z:3d}          {sigmoid(z):.4f}")
16     print()
17
18     # Hai tính chất cần tự kiểm chứng
19     print("Kiểm tra sigmoid(-z) = 1 - sigmoid(z):")
20     for z in [1.0, 2.5, 3.7]:
21         trai = sigmoid(-z)
22         phai = 1 - sigmoid(z)
23         print(f"  z = {z}:  {trai:.6f}  va  {phai:.6f}")
24     print()
25
26     # Thử với một mô hình giả định w = 0.4 và b = -5
27     w, b = 0.4, -5.0
28     print(f"Mo hình giả định: w = {w}, b = {b}")
29     print(" So gio on      z = w*x + b      Xac suat qua mon")
30     for gio in [5, 10, 12.5, 15, 20, 25]:
31         z = w * gio + b
32         print(f"  {gio:5.1f}          {z:6.2f}          {sigmoid(z):.4f}")

```

Đoạn mã làm ba việc. Việc thứ nhất là in bảng giá trị để sinh viên thấy hình dạng của hàm bằng con số thay vì bằng hình. Việc thứ hai là kiểm chứng một tính chất đẹp, $\sigma(-z) = 1 - \sigma(z)$. Việc thứ ba là thử một mô hình giả định với $w = 0.4$ và $b = -5$ để thấy sigmoid được dùng như thế nào.

python code/c2_sigmoid.py

Bảng giá trị của hàm sigmoid

z	sigmoid(z)
-6	0.0025
-4	0.0180
-2	0.1192
-1	0.2689
0	0.5000
1	0.7311
2	0.8808
4	0.9820
6	0.9975

Kiểm tra sigmoid(-z) = 1 - sigmoid(z):

z = 1.0:	0.268941	va	0.268941
z = 2.5:	0.075858	va	0.075858
z = 3.7:	0.024127	va	0.024127

Mô hình giả định: w = 0.4, b = -5.0

So gio on	z = w*x + b	Xac suat qua mon
5.0	-3.00	0.0474
10.0	-1.00	0.2689
12.5	0.00	0.5000
15.0	1.00	0.7311

20.0	3.00	0.9526
25.0	5.00	0.9933

Đọc bảng thứ nhất từ trên xuống, giá trị đi từ 0.0025 lên 0.9975 nhưng không bao giờ chạm 0 hay chạm 1. Đó là điều đáng nhớ: mô hình logistic không bao giờ nói chắc chắn tuyệt đối, và điều này hợp lý vì thực tế cũng không có gì chắc chắn tuyệt đối.

Bảng thứ hai xác nhận $\sigma(-z) = 1 - \sigma(z)$ tới sáu chữ số thập phân. Tính chất này có ý nghĩa rất đời thường: xác suất qua môn cộng xác suất rớt môn luôn bằng 1.

Bảng thứ ba là chỗ quan trọng nhất. Với $w = 0.4$ và $b = -5$, một bạn ôn 12.5 giờ cho z bằng đúng 0, nên xác suất qua môn bằng đúng 0.5. Bạn ôn 5 giờ chỉ được 0.0474, còn bạn ôn 25 giờ được tới 0.9933.

Hai bước luôn đi cùng nhau

Mô hình logistic làm đúng hai bước. Bước một tính phần tuyến tính $z = wx + b$, y hết bài 1. Bước hai bọc sigmoid lên trên để ép z về khoảng 0 tới 1. Vì phần tuyến tính giữ nguyên nên tên gọi mới có chữ *hồi quy*, dù đây là bài toán phân loại.

Chú ý dấu trừ trong công thức

Công thức là $1/(1 + e^{-z})$, mũ mang dấu trừ. Trong numpy phải viết `np.exp(-z)` chứ không phải `np.exp(z)`. Viết nhầm dấu thì đồ thị lật ngược, và mô hình sẽ đoán ngược hoàn toàn mà không báo lỗi gì cả. Đây là loại lỗi khó phát hiện nhất, vì chương trình vẫn chạy trơn tru.

4. Khớp mô hình bằng scikit-learn

4.1. Ba bước quen thuộc

Ở bài 1 sinh viên đã học ba bước chuẩn của scikit-learn. Bài này dùng lại đúng ba bước đó, chỉ đổi tên lớp mô hình.

Bảng 3: Ba bước chuẩn, so sánh giữa hai bài.

Bước	Bài 1	Bài 2
1	<code>LinearRegression()</code>	<code>LogisticRegression()</code>
2	<code>.fit(X, y)</code>	<code>.fit(X, y)</code> giống hệt
3	<code>.predict(X_moi)</code>	<code>.predict</code> hoặc <code>.predict_proba</code>

Chỉ có bước 3 là thêm lựa chọn mới. Hàm `predict_proba` trả về xác suất, còn `predict` trả thẳng nhãn 0 hoặc 1. Mục 6 sẽ cho thấy vì sao nhiều khi ta cần xác suất chứ không cần nhãn.

```

1 # -*- coding: utf-8 -*-
2 """Bước 3: để scikit-learn tìm w và b từ dữ liệu thật."""
3
4 import pandas as pd
5 from sklearn.linear_model import LogisticRegression
6

```

```

7 df = pd.read_csv("data/sinh_vien.csv")
8
9 # X phải là bảng hai chiều nên viết hai cặp ngoặc vuông.
10 # y là dãy một chiều nên chỉ một cặp.
11 X = df[["gio_on"]]
12 y = df["qua_mon"]
13
14 mo_hinh = LogisticRegression()
15 mo_hinh.fit(X, y)
16
17 w = float(mo_hinh.coef_[0][0])
18 b = float(mo_hinh.intercept_[0])
19
20 print(f"He so goc w = {w:.6f}")
21 print(f"He so chan b = {b:.6f}")
22 print()
23
24 # Số giờ ôn mà mô hình phân vân đúng năm mươi năm mươi
25 print(f"So gio on ung voi xac suat 0.5: {-b / w:.2f} gio")
26 print()
27
28 can_moi = pd.DataFrame({"gio_on": [5.0, 10.0, 13.0, 20.0, 28.0]})
29 xac_suat = mo_hinh.predict_proba(can_moi)[: , 1]
30 nhan = mo_hinh.predict(can_moi)
31
32 print("Du doan cho nam ban moi:")
33 print(" So gio on      Xac suat qua      Nhan mo hinh dua ra")
34 for gio, p, n in zip(can_moi["gio_on"], xac_suat, nhan):
35     print(f" {gio:5.1f}          {p:.4f}          {n}")

```

Hình dạng dữ liệu, chỗ sai kinh điển

Biến X viết là `df[["gio_on"]]` với HAI cặp ngoặc vuông, còn y viết là `df["qua_mon"]` với MỘT cặp. Lý do y như bài 1: scikit-learn cần X là bảng hai chiều còn y là dãy một chiều. Viết nhầm một cặp cho X thì chương trình dừng với thông báo `Expected a 2-dimensional container but got pandas.Series`.

python code/c3_khop_mo_hinh.py

```

He so goc w = 0.392819
He so chan b = -5.064890

So gio on ung voi xac suat 0.5: 12.89 gio

Du doan cho nam ban moi:
So gio on      Xac suat qua      Nhan mo hinh dua ra
5.0           0.0431           0
10.0          0.2429           0

```

13.0	0.5104	1
20.0	0.9422	1
28.0	0.9974	1

4.2. Đọc kết quả

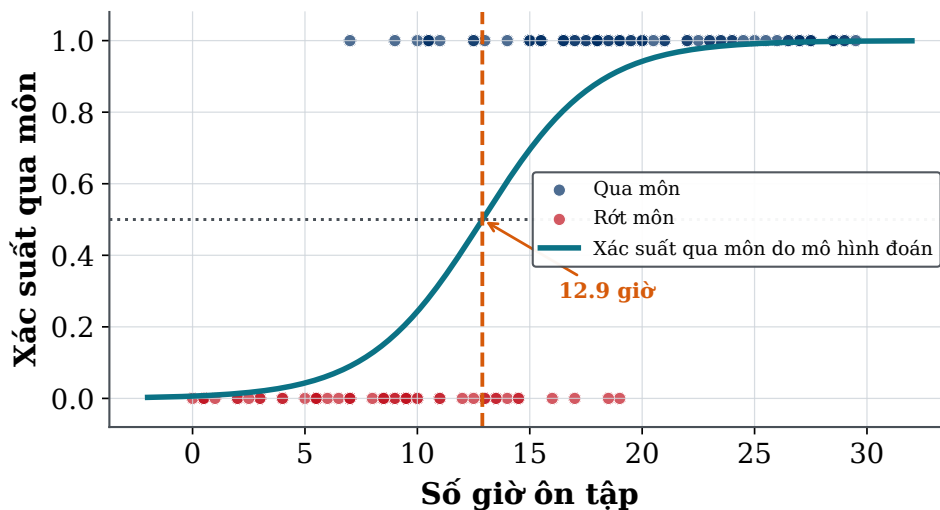
Máy tìm được $w = 0.392819$ và $b = -5.064890$. Mô hình đầy đủ vì vậy là

$$\hat{y} = \sigma(0.3928x - 5.0649)$$

với x là số giờ ôn tập và $\hat{y}$ là xác suất qua môn.

Dấu của w là dương, nghĩa là ôn càng nhiều thì xác suất qua càng cao. Nếu chạy ra w âm thì gần như chắc chắn có gì đó sai, vì kết quả đó đi ngược mọi hiểu biết thông thường.

Con số đáng nhớ nhất là 12.89 giờ. Đó là số giờ ôn mà mô hình phân vân đúng năm mươi năm mươi, tìm được bằng cách giải $w x + b = 0$, tức $x = -b/w$. Dưới mốc này mô hình nghiêng về rớt, trên mốc này mô hình nghiêng về qua.



Hình 3: Đường cong logistic khớp trên 120 sinh viên. Đường thẳng đứng đánh dấu mốc 12.89 giờ, nơi xác suất bằng đúng 0.5.

Bảng dự đoán cho năm bạn mới cho thấy đường cong hoạt động ra sao. Bạn ôn 5 giờ chỉ được 0.0431, gần như chắc rớt. Bạn ôn 13 giờ được 0.5104, tức nhỉnh hơn phân vân một chút, và đúng như dự đoán vì 13 giờ chỉ nhiều hơn mốc 12.89 giờ có một chút. Bạn ôn 28 giờ được 0.9974, gần như chắc qua.

Đừng đọc w như hệ số của bài 1

Ở hồi quy tuyến tính, w là mức tăng của đầu ra khi x tăng một đơn vị. Ở hồi quy logistic, w là mức tăng của z , **không phải** của xác suất. Ôn thêm một giờ không làm xác suất tăng thêm 0.3928. Xác suất tăng nhiều hay ít còn tùy đang đứng ở chỗ nào trên đường cong chữ S. Ở giữa đường cong, quanh mốc 12.89 giờ, thêm một giờ làm xác suất nhích khá mạnh. Ở hai đầu, khi xác suất đã gần 0 hoặc gần 1, thêm một giờ gần như không đổi gì.

5. Chấm điểm mô hình

5.1. Ma trận nhầm lẫn

Với bài toán hai nhãn, mỗi dự đoán rơi vào đúng một trong bốn ô. Bảng gom bốn ô đó gọi là ma trận nhầm lẫn (*confusion matrix*).

Bảng 4: Bốn kiểu kết quả của một bộ phân loại nhị phân.

Viết tắt	Tên đầy đủ	Nghĩa trong bài này
TP	True Positive	Đoán qua, thật sự qua. Đoán đúng.
TN	True Negative	Đoán rớt, thật sự rớt. Đoán đúng.
FP	False Positive	Đoán qua, nhưng thật ra rớt. Đoán sai.
FN	False Negative	Đoán rớt, nhưng thật ra qua. Đoán sai.

Cách nhớ bốn chữ viết tắt

Chữ thứ hai cho biết mô hình đoán gì. P là positive, tức mô hình đoán lớp 1. N là negative, tức mô hình đoán lớp 0.

Chữ thứ nhất cho biết đoán đó đúng hay sai. T là true, tức đoán đúng. F là false, tức đoán sai.

Ghép lại thì FP đọc là *đoán lớp 1 nhưng sai*, nghĩa là thật ra nó thuộc lớp 0.

5.2. Chia dữ liệu rời chấm điểm

Quy tắc của bài 1 vẫn nguyên giá trị: học trên dữ liệu nào thì không được chấm điểm trên dữ liệu đó. Điểm mới ở bài này là tham số `stratify`.

```

1  # -*- coding: utf-8 -*-
2  """Bước 4: chia dữ liệu rời chấm điểm mô hình bằng bốn thước đo."""
3
4  import pandas as pd
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.metrics import (accuracy_score, confusion_matrix, f1_score,
7                               precision_score, recall_score)
8  from sklearn.model_selection import train_test_split
9
10 df = pd.read_csv("data/sinh_vien.csv")
11 X = df[["gio_on"]]
12 y = df["qua_mon"]
13
14 # stratify=y giữ đúng tỷ lệ qua và rớt ở cả hai phần sau khi chia
15 X_train, X_test, y_train, y_test = train_test_split(
16     X, y, test_size=0.25, random_state=17, stratify=y
17 )
18
19 print("Số sinh viên de hoc      :", len(X_train))
20 print("Số sinh viên de kiểm tra:", len(X_test))
21 print()
22
23 mo_hinh = LogisticRegression()
24 mo_hinh.fit(X_train, y_train)
25

```

```

26 y_pred = mo_hinh.predict(X_test)
27
28 # Thứ tự bốn ô do scikit-Learn quy định là TN, FP, FN, TP
29 tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
30 print("Ma tran nham lan")
31 print(f"  TN = {tn:2d}    doan rot, that su rot")
32 print(f"  FP = {fp:2d}    doan qua, that ra rot")
33 print(f"  FN = {fn:2d}    doan rot, that ra qua")
34 print(f"  TP = {tp:2d}    doan qua, that su qua")
35 print()
36
37 print(f"Accuracy  = {accuracy_score(y_test, y_pred):.4f}")
38 print(f"Precision = {precision_score(y_test, y_pred):.4f}")
39 print(f"Recall    = {recall_score(y_test, y_pred):.4f}")
40 print(f"F1       = {f1_score(y_test, y_pred):.4f}")
41 print()
42
43 # Tự tính lại bằng tay để đối chiếu với thư viện
44 print("Tu tinh lai bang cong thuc:")
45 n = len(y_test)
46 print(f"  Accuracy  = ({tp} + {tn}) / {n} = {(tp + tn) / n:.4f}")
47 print(f"  Precision = {tp} / ({tp} + {fp}) = {tp / (tp + fp):.4f}")
48 print(f"  Recall   = {tp} / ({tp} + {fn}) = {tp / (tp + fn):.4f}")

```

Vì sao cần

Bộ dữ liệu có 71 bạn qua và 49 bạn rớt, tức tỷ lệ qua khoảng 59 phần trăm. Nếu chia ngẫu nhiên tự do, tập kiểm tra có thể vô tình chứa toàn bạn qua môn, và khi đó không chấm điểm được gì cả. Tham số `stratify=y` buộc máy giữ đúng tỷ lệ 59 phần trăm đó ở cả tập học lẫn tập kiểm tra. Với bài phân loại, gần như lúc nào cũng nên đặt tham số này.

python code/c4_danh_gia.py

```

So sinh vien de hoc      : 90
So sinh vien dekiem tra: 30

Ma tran nham lan
  TN =  9   doan rot, that su rot
  FP =  3   doan qua, that ra rot
  FN =  1   doan rot, that ra qua
  TP = 17   doan qua, that su qua

Accuracy  = 0.8667
Precision = 0.8500
Recall    = 0.9444
F1        = 0.8947

Tu tinh lai bang cong thuc:

```

```

Accuracy = (17 + 9) / 30 = 0.8667
Precision = 17 / (17 + 3) = 0.8500
Recall    = 17 / (17 + 1) = 0.9444

```

Thật: tốt	9 TN	3 FP
Thật: qua	1 FN	17 TP
	Đ đoán tốt	Đ đoán qua

Hình 4: Ma trận nhầm lẫn trên 30 sinh viên của tập kiểm tra. Hai ô trên đường chéo là số lần đoán đúng.

Trong 30 bạn của tập kiểm tra, mô hình đoán đúng 26 bạn và sai 4 bạn. Bốn bạn sai chia làm hai loại, 3 bạn kiểu FP và 1 bạn kiểu FN.

5.3. Bốn thước đo

Bốn công thức đều tính từ bốn con số trên

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Đọc từng con số bằng lời thì dễ nhớ hơn nhiều.

- **Accuracy** bằng 0.8667, nghĩa là mô hình đoán đúng khoảng 87 phần trăm số bạn.
- **Precision** bằng 0.8500, nghĩa là trong 20 bạn mà mô hình nói sẽ qua, có 17 bạn qua thật, tức 85 phần trăm.
- **Recall** bằng 0.9444, nghĩa là trong 18 bạn thật sự qua môn, mô hình tìm ra được 17 bạn.
- F_1 bằng 0.8947, là con số gộp precision và recall lại cho tiện so sánh.

Mẹo phân biệt precision và recall

Cả hai đều có TP ở tử số, chỉ khác mẫu số.

Precision chia cho **những gì mô hình nói**, tức $TP + FP$.

Recall chia cho **những gì có thật**, tức $TP + FN$.

Ở đây recall 0.9444 cao hơn precision 0.8500, nghĩa là mô hình hơi dễ tính. Nó bắt được gần hết các bạn qua môn, nhưng đổi lại báo nhầm 3 bạn.

Accuracy một mình có thể đánh lừa

Giả sử một lớp có 100 sinh viên mà chỉ 5 bạn rớt. Một mô hình lười biếng cứ đoán mọi bạn đều qua sẽ đạt accuracy bằng 0.95, nghe rất giỏi. Nhưng nó không bắt được bạn nào trong 5 bạn cần cảnh báo.

Chú ý một chi tiết quan trọng. Nếu vẫn lấy lớp dương là qua môn thì recall của mô hình lười đó bằng 1.00, nghe càng đẹp. Phải đổi lớp dương sang lớp rớt môn mới lòi ra sự thật, khi đó recall bằng 0.

Vì vậy mỗi lần báo cáo một con số, luôn phải nói rõ đang tính cho lớp nào.

6. Ngưỡng quyết định

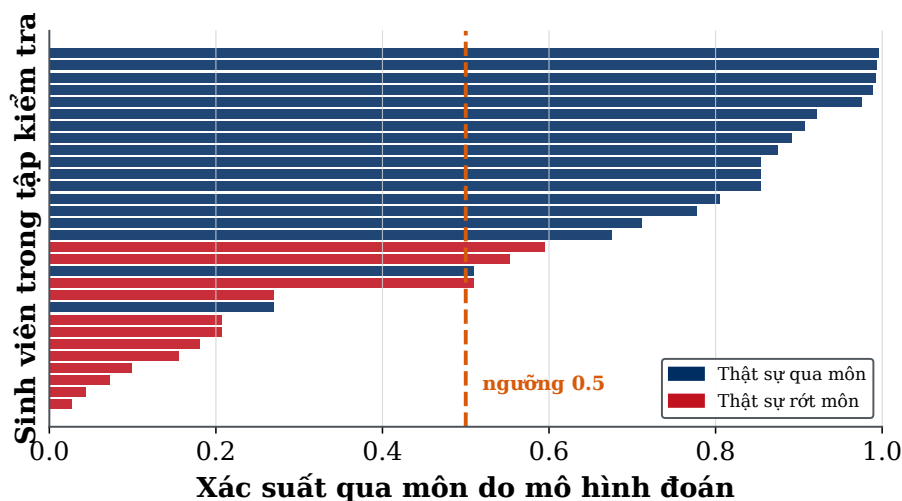
6.1. Mô hình trả về xác suất, không phải nhãn

Hàm `predict` rất tiện nhưng nó giấu mất một bước. Thực ra mô hình chỉ trả về một xác suất, rồi `predict` tự so xác suất đó với 0.5 để ra nhãn. Con số 0.5 gọi là ngưỡng (*threshold*), và nó là lựa chọn mặc định chứ không phải chân lý.

Từ xác suất ra nhãn

$$\hat{c} = 1 \text{ nếu } \hat{y} \geq t, \quad \hat{c} = 0 \text{ nếu } \hat{y} < t$$

Ở đây $\hat{y}$ là xác suất mô hình trả về, $\hat{c}$ là nhãn cuối cùng, còn t là ngưỡng do ta chọn.



Hình 5: Xác suất mà mô hình gán cho từng bạn trong tập kiểm tra, xếp theo thứ tự tăng dần. Màu thanh cho biết sự thật, đường kẻ dọc là ngưỡng 0.5.

Nhìn hình 5 sẽ thấy vài thanh màu đỏ nằm bên phải đường kẻ, đó chính là 3 bạn kiểu FP. Nếu đẩy đường kẻ sang phải, tức nâng ngưỡng, thì mấy thanh đỏ đó sẽ rơi về phía trái và hết bị đoán nhầm. Nhưng đồng thời vài thanh xanh cũng rơi theo, và chúng trở thành lỗi kiểu FN. Đó chính là sự đánh đổi mà mục này bàn tới.

6.2. Tự áp ngưỡng

```

1  # -*- coding: utf-8 -*-
2  """Bước 5: tự áp ngưỡng và xem precision với recall đổi ra sao."""
3
4  import pandas as pd
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.metrics import precision_score, recall_score
7  from sklearn.model_selection import train_test_split
8
9  df = pd.read_csv("data/sinh_vien.csv")
10 X = df[["gio_on"]]
11 y = df["qua_mon"]
12
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=0.25, random_state=17, stratify=y
15 )
16 mo_hinh = LogisticRegression().fit(X_train, y_train)
17
18 # predict_proba trả về hai cột, cột 0 cho Lớp 0 và cột 1 cho Lớp 1
19 p = mo_hinh.predict_proba(X_test)[: , 1]
20
21 print("Nguong    So ban bi doan la qua    Precision    Recall")
22 for nguong in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
23     y_pred = (p >= nguong).astype(int)
24     pre = precision_score(y_test, y_pred, zero_division=1)
25     rec = recall_score(y_test, y_pred, zero_division=0)
26     print(f" {nguong:.1f}           {y_pred.sum():3d}"
27           f"           {pre:.4f}           {rec:.4f}")
28 print()
29
30 print("Doc bang tren tu duoi len:")
31 print("    Nguong cang cao thi mo hinh cang kho tinh,")
32 print("    precision tang nhung recall giam. Nguong thap thi nguoc lai.")

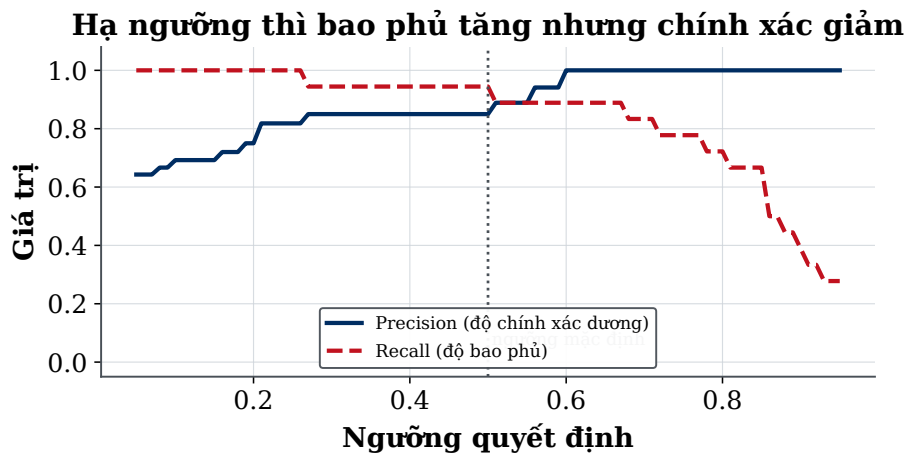
```

python code/c5_doi_nguong.py

Nguong	So ban bi doan la qua	Precision	Recall
0.2	24	0.7500	1.0000
0.3	20	0.8500	0.9444
0.4	20	0.8500	0.9444
0.5	20	0.8500	0.9444
0.6	16	1.0000	0.8889
0.7	15	1.0000	0.8333
0.8	13	1.0000	0.7222

Doc bang tren tu duoi len:

Nguong cang cao thi mo hinh cang kho tinh,
precision tang nhung recall giam. Nguong thap thi nguoc lai.



Hình 6: Precision và recall khi ngưỡng chạy từ 0.05 tới 0.95. Hai đường đi ngược chiều nhau.

Bảng trên cho thấy quy luật rất rõ.

- Ở ngưỡng 0.2, mô hình gắn nhãn qua cho 24 bạn. Recall đạt 1.0000, tức không bỏ sót bạn nào thật sự qua, nhưng precision chỉ còn 0.7500.
- Ở ngưỡng 0.5, mô hình gắn nhãn qua cho 20 bạn. Precision 0.8500 và recall 0.9444.
- Ở ngưỡng 0.8, mô hình chỉ còn gắn nhãn qua cho 13 bạn. Precision đạt 1.0000, tức không báo nhầm bạn nào, nhưng recall tụt xuống 0.7222.

Quy luật phải nhớ

Hạ ngưỡng thì mô hình dễ tính hơn, nó gắn nhãn 1 cho nhiều người hơn, nên **recall tăng** còn **precision giảm**.

Nâng ngưỡng thì ngược lại, mô hình khó tính hơn, **precision tăng** còn **recall giảm**.

Không có ngưỡng nào làm cả hai cùng tốt lên.

6.3. Chọn ngưỡng theo bài toán

Vì không có ngưỡng tốt nhất một cách tuyệt đối, ta phải chọn theo cái giá phải trả của từng loại sai.

Bảng 5: Ba tình huống thực tế và ngưỡng nên chọn cho từng tình huống.

Bài toán	Loại sai đắt hơn	Ngưỡng nên chọn
Cảnh báo sinh viên sắp rớt, nhãn 1 là qua môn	FP, tức báo một bạn sẽ qua trong khi bạn đó sắp rớt, nên bỏ sót mất	Nâng ngưỡng, cho mô hình khó gắn nhãn qua hơn
Sàng lọc một bệnh chữa được nếu phát hiện sớm, nhãn 1 là có bệnh	FN, tức bỏ sót một người có bệnh thật	Hạ ngưỡng, cho mô hình dễ báo bệnh hơn
Lọc thư rác và xoá thẳng, nhãn 1 là thư rác	FP, tức xoá nhầm một lá thư quan trọng	Nâng ngưỡng, thà để lọt thư rác còn hơn mất thư thật

Câu hỏi phải tự trả lời trước khi chọn ngưỡng

Lớp tôi quan tâm là lớp nào, và sai kiểu nào đắt hơn?

Chú ý cả hai câu đều quan trọng. Cùng một ngưỡng cao có thể là đúng ở bài này và sai ở bài kia, tùy nhãn 1 được đặt cho lớp nào. Ba dòng trong bảng 5 cho thấy điều đó rất rõ.

7. Thêm biến đầu vào thứ hai

7.1. Trong mã chỉ đổi đúng một chỗ

Kết quả cuối môn đầu chỉ phụ thuộc số giờ ôn. Điểm giữa kỳ cũng nói lên nhiều điều, vì nó phản ánh cả quá trình học trước đó. Mô hình hai biến có dạng

$$\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$$

với x_1 là số giờ ôn và x_2 là điểm giữa kỳ.

```

1  # -*- coding: utf-8 -*-
2  """Bước 6: thêm điểm giữa kỳ làm biến thứ hai."""
3
4  import pandas as pd
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.metrics import accuracy_score
7  from sklearn.model_selection import train_test_split
8
9  df = pd.read_csv("data/sinh_vien.csv")
10 y = df["qua_mon"]
11
12 # Chỉ đổi đúng một chỗ: danh sách cột đưa vào X
13 for ten, cot in [("Mot bien", ["gio_on"]),
14                 ("Hai bien", ["gio_on", "diem_giua_ky"])]:
15     X = df[cot]
16     X_train, X_test, y_train, y_test = train_test_split(
17         X, y, test_size=0.25, random_state=17, stratify=y
18     )
19     mo_hinh = LogisticRegression().fit(X_train, y_train)
20     do_chinh_xac = accuracy_score(y_test, mo_hinh.predict(X_test))
21     print(f"{ten}: accuracy tren tap kiem tra = {do_chinh_xac:.4f}")
22 print()
23
24 # Xem kỹ hệ số của mô hình hai biến
25 X = df[["gio_on", "diem_giua_ky"]]
26 X_train, X_test, y_train, y_test = train_test_split(
27     X, y, test_size=0.25, random_state=17, stratify=y
28 )
29 mo_hinh = LogisticRegression().fit(X_train, y_train)
30
31 print("He so cua tung bien:")
32 for ten_cot, he_so in zip(X.columns, mo_hinh.coef_[0]):
33     print(f" {ten_cot:14s} {he_so:+.4f}")
34 print(f" {'he so chan':14s} {mo_hinh.intercept_[0]:+.4f}")

```

```

35 print()
36 print("Ca hai he so deu duong, nghĩa là ôn nhiều hơn và")
37 print("diem giua ky cao hơn đều làm tăng xác suất qua môn.")

```

Điểm hay của scikit-learn

Chuyển từ một biến sang hai biến chỉ là thêm tên cột vào danh sách trong ngoặc vuông. Mọi dòng còn lại giữ nguyên hoàn toàn. Đây chính là lợi ích của việc mọi mô hình dùng chung một giao diện, và nó đúng cho cả những mô hình phức tạp hơn ở các bài sau.

python code/c6_hai_bien.py

```

Mot bien: accuracy tren tap kiem tra = 0.8667
Hai bien: accuracy tren tap kiem tra = 0.9000

He so cua tung bien:
gio_on      +0.3035
diem_giua_ky +0.5885
he so chan  -6.9811

Ca hai he so deu duong, nghĩa là ôn nhiều hơn và
diem giua ky cao hơn đều làm tăng xác suất qua môn.

```

7.2. Đọc kết quả

Accuracy tăng từ 0.8667 lên 0.9000. Mức tăng này có thật nhưng không lớn, vì số giờ ôn đã giải thích được phần lớn kết quả rồi. Trên 30 bạn kiểm tra, mô hình hai biến đoán đúng thêm được 1 bạn so với mô hình một biến.

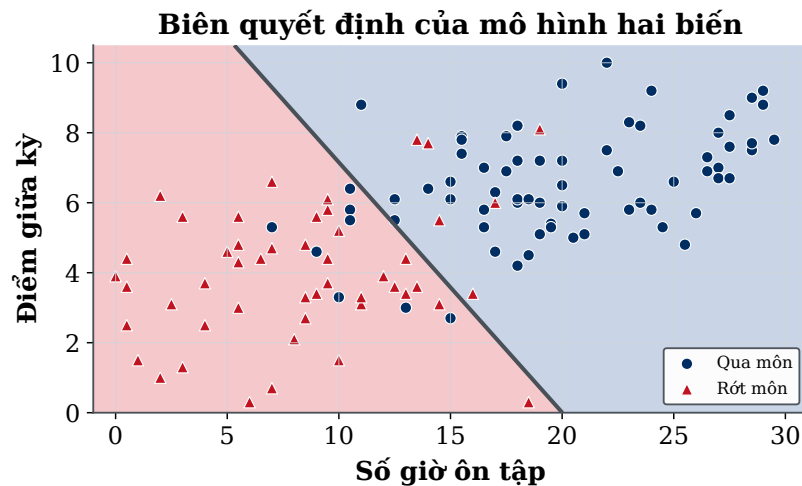
Đừng vội mừng với mức tăng nhỏ

Tập kiểm tra chỉ có 30 bạn, nên chênh lệch 1 bạn đã làm accuracy đổi hơn 3 phần trăm. Với cỡ mẫu nhỏ như vậy, mức tăng từ 0.8667 lên 0.9000 chưa đủ để kết luận chắc chắn mô hình hai biến tốt hơn. Muốn kết luận chắc, phải chia dữ liệu nhiều lần rồi lấy trung bình, kỹ thuật đó gọi là kiểm định chéo (*cross validation*) và sinh viên sẽ học ở bài sau.

Hai hệ số đều mang dấu dương, $w_1 = 0.3035$ cho số giờ ôn và $w_2 = 0.5885$ cho điểm giữa kỳ. Dấu dương khớp với trực giác: ôn nhiều hơn và điểm giữa kỳ cao hơn đều làm tăng khả năng qua môn.

Đừng so độ lớn hai hệ số với nhau

Nhìn qua rất dễ kết luận điểm giữa kỳ quan trọng gần gấp đôi số giờ ôn, vì 0.5885 gần gấp đôi 0.3035. Kết luận đó **không đúng**, vì hai biến có thang đo khác nhau hoàn toàn. Số giờ ôn chạy từ 0 tới 29.5, còn điểm giữa kỳ chỉ chạy từ 0 tới 10. Tăng một giờ ôn và tăng một điểm giữa kỳ là hai chuyện có độ khó rất khác nhau. Muốn so sánh được thì phải đưa hai biến về cùng thang đo trước, bằng `StandardScaler`, và đó là nội dung của những bài sau.



Hình 7: Biên quyết định của mô hình hai biến. Vùng xanh là nơi mô hình đoán qua môn, vùng hồng là nơi mô hình đoán rớt.

Hình 7 cho thấy một điều quan trọng về hồi quy logistic. Ranh giới giữa hai vùng là một đường **thẳng**, không phải đường cong. Tập hợp các điểm mà mô hình phân vân đúng 50 phần trăm gọi là biên quyết định (*decision boundary*).

Biên quyết định của logistic luôn thẳng

Với một biến đầu vào, biên là một điểm, chính là mốc 12.89 giờ ở mục 4. Với hai biến, biên là một đường thẳng như trong hình. Với ba biến trở lên, biên là một mặt phẳng. Điểm chung là biên không bao giờ uốn cong. Đó vừa là điểm mạnh, vì mô hình đơn giản và khó bị quá khớp, vừa là giới hạn, vì có những bộ dữ liệu mà một đường thẳng không tài nào chia được. Các bài sau sẽ giới thiệu những mô hình vẽ được biên cong.

8. Bài tập

Sinh viên tạo thư mục `baitap02` nằm ngay trong thư mục bài thực hành, cạnh thư mục `data` và `code`. Mỗi bài viết thành một tệp Python riêng, đặt tên theo số bài, ví dụ `bai1.py`.

Quy tắc đường dẫn giữ nguyên như bài 1. Sinh viên luôn chạy lệnh từ thư mục gốc, tức gõ `python baitap02/bai1.py`, và trong mã vẫn viết `data/sinh_vien.csv`. Nhờ vậy chép tệp mẫu từ `code` sang `baitap02` là chạy được ngay.

Bài nộp gồm tệp mã nguồn và ảnh chụp màn hình phần kết quả in ra.

Bài tập 1: Thống kê theo nhóm

Đọc bộ dữ liệu rồi in ra ba con số. Số bạn có điểm giữa kỳ từ 7 trở lên, tỷ lệ qua môn của riêng nhóm đó, và tỷ lệ qua môn của nhóm còn lại.

Gợi ý, dùng phép lọc theo điều kiện của `pandas` rồi lấy `.mean()` của cột `qua_mon`. Nhận xét một câu về việc điểm giữa kỳ có phân biệt được hai nhóm hay không.

Bài tập 2: Vẽ hàm sigmoid

Vẽ đồ thị hàm sigmoid với z chạy từ -8 tới 8 , lưu thành tệp `sigmoid.png`. Trên cùng hình, vẽ thêm một đường ngang tại mức 0.5 và một đường dọc tại $z = 0$.

Gợi ý, dùng `np.linspace`, `plt.plot`, `plt.axhline`, `plt.axvline` và `plt.savefig`.

Bài tập 3: Dự đoán cho một bạn cụ thể

Khớp lại mô hình một biến như ở mục 4, rồi viết một hàm `du_doan(gio)` nhận vào số giờ ôn và in ra ba thứ. Giá trị z , xác suất qua môn, và nhãn theo ngưỡng 0.5. Gọi hàm đó với các giá trị 3, 8, 12.89, 18 và 26 giờ. Giải thích vì sao với 12.89 giờ thì xác suất ra gần đúng 0.5.

Bài tập 4: Tự tính bốn thước đo

Chép tệp `c4_danh_gia.py` sang `baitap02` rồi bỏ hết các hàm `accuracy_score`, `precision_score`, `recall_score`, `f1_score` của `scikit-learn` đi. Tự tính bốn con số đó chỉ từ bốn số TP, TN, FP, FN bằng công thức ở mục 5. So kết quả tự tính với kết quả thư viện in ra trong tài liệu, hai bên phải khớp. Nộp cả phần mã tự tính và ảnh chụp kết quả.

Bài tập 5: Dò ngưỡng tốt nhất theo F_1

Cho ngưỡng chạy từ 0.05 tới 0.95, mỗi bước 0.05. Với mỗi ngưỡng, tính F_1 trên tập kiểm tra rồi in ra thành bảng. Cuối cùng in ra ngưỡng cho F_1 cao nhất. Nhận xét một câu: ngưỡng tốt nhất theo F_1 có đúng bằng 0.5 không? Gợi ý, dùng `np.arange(0.05, 1.0, 0.05)` và hàm `f1_score` với tham số `zero_division=0`.

Bài tập 6: Đổi lớp dương rồi chấm lại

Ở mục 5 ta luôn lấy lớp dương là qua môn. Bây giờ hãy chấm điểm cho **lớp rớt môn**, tức lớp 0. Gợi ý, thêm tham số `pos_label=0` vào `precision_score` và `recall_score`. In ra precision và recall của lớp 0 rồi so với con số của lớp 1 trong tài liệu. Giải thích bằng lời vì sao hai bộ số khác nhau, dù mô hình và dữ liệu không đổi chút nào.

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, và tên biến đặt có nghĩa. Riêng bài 4 và bài 6 chấm nặng phần giải thích, vì đó mới là chỗ kiểm tra sinh viên có thật sự hiểu bốn thước đo hay không.

9. Phụ lục A, bảng tra nhanh

Bảng 6: Các hàm mới xuất hiện trong bài 2, ngoài những hàm đã học ở bài 1.

Lệnh	Thư viện	Công dụng
LogisticRegression	scikit-learn	Tạo mô hình hồi quy logistic chưa huấn luyện.
.predict_proba	scikit-learn	Trả về xác suất của từng lớp, cột 1 là lớp dương.
confusion_matrix	scikit-learn	Trả về ma trận nhầm lẫn theo thứ tự TN, FP, FN, TP.
.ravel()	numpy	Trải ma trận hai chiều thành dãy một chiều để tách bốn số.
accuracy_score	scikit-learn	Tỷ lệ dự đoán đúng trên tổng số.
precision_score	scikit-learn	Trong số được gán nhãn dương, bao nhiêu phần đúng.
recall_score	scikit-learn	Trong số thật sự dương, bắt được bao nhiêu phần.
f1_score	scikit-learn	Gộp precision và recall thành một con số.
classification_report	scikit-learn	In một lượt cả bốn thước đo cho từng lớp.
stratify=y	scikit-learn	Giữ nguyên tỷ lệ các lớp khi chia dữ liệu.
pos_label	scikit-learn	Chỉ định lớp nào được coi là lớp dương.
.value_counts()	pandas	Đếm số lần xuất hiện của từng giá trị trong một cột.
.groupby()	pandas	Tách dữ liệu thành nhóm theo giá trị của một cột.
np.exp	numpy	Tính e mũ, dùng trong công thức sigmoid.

Bảng 7: Ký hiệu toán dùng trong bài 2.

Ký hiệu	Ý nghĩa
y	Nhãn thật, chỉ nhận 0 hoặc 1. Ở bài này 1 là qua môn.
$\hat{y}$	Xác suất qua môn mà mô hình trả về, một số trong khoảng 0 tới 1.
$\hat{c}$	Nhãn cuối cùng mà mô hình đưa ra, chỉ nhận 0 hoặc 1.
z	Phần tuyến tính $wx + b$, nhận giá trị thực bất kỳ.
σ	Hàm sigmoid, ép z về khoảng 0 tới 1.
t	Ngưỡng quyết định, mặc định bằng 0.5.
w, b	Hệ số góc và hệ số chặn, hai con số mà máy học được.

Bảng 8: Các con số kết quả chính của bài thực hành.

Đại lượng	Giá trị
Số sinh viên trong bộ dữ liệu	120, gồm 71 qua và 49 rớt
Số giờ ôn trung bình, nhóm rớt và nhóm qua	8.29 và 19.89 giờ
Hệ số của mô hình một biến	$w = 0.392819$, $b = -5.064890$
Số giờ ôn ứng với xác suất 0.5	12.89 giờ
Chia dữ liệu	90 để học, 30 để kiểm tra
Ma trận nhầm lẫn	TP = 17, TN = 9, FP = 3, FN = 1
Accuracy, Precision	0.8667 và 0.8500
Recall, F_1	0.9444 và 0.8947
Accuracy của mô hình hai biến	0.9000

10. Phụ lục B, lỗi thường gặp

Bảng 9: Các lỗi hay gặp riêng của bài hồi quy logistic.

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
FileNotFoundError, không tìm thấy data/sinh_vien.csv	Đang chạy Python từ bên trong thư mục code hoặc baitap02. Hãy quay về thư mục gốc của bài rồi chạy lại.
ValueError: Expected a 2-dimensional container but got pandas.Series predict_proba trả về bảng hai cột, không biết lấy cột nào Xác suất in ra toàn 0 hoặc toàn 1	Viết <code>df["gio_on"]</code> với một cặp ngoặc vuông cho X. Phải viết <code>df[["gio_on"]]</code> với hai cặp. Cột 0 là xác suất của lớp 0, cột 1 là xác suất của lớp 1. Hầu như luôn lấy cột 1, tức viết <code>[:, 1]</code> . Nhiều khả năng viết nhầm <code>np.exp(z)</code> thành <code>np.exp(-z)</code> hoặc ngược lại trong hàm sigmoid. Kiểm lại bằng cách tính <code>sigmoid(0)</code> , kết quả phải đúng bằng 0.5.
Mô hình cho w mang dấu âm	Có thể đã đổi nhầm nhãn, hoặc đưa nhầm cột vào X. In <code>df.head()</code> ra kiểm tra lại tên cột và giá trị nhãn.
ConvergenceWarning: lbfgs failed to converge	Thuật toán chưa hội tụ trong số vòng mặc định. Với dữ liệu nhiều biến, hãy chuẩn hóa bằng <code>StandardScaler</code> , hoặc tăng <code>max_iter</code> .
ValueError: pos_label=1 is not a valid label	Cột nhãn không chứa giá trị 1, ví dụ nhãn đang là chuỗi "qua" và "rớt". Đổi nhãn sang 0 và 1 trước, hoặc truyền đúng <code>pos_label</code> .
Precision bằng 1.0 mà recall rất thấp	Ngưỡng đang quá cao nên mô hình hầu như không gắn nhãn dương cho ai. Đây không phải lỗi, chỉ là hệ quả của ngưỡng. Xem lại mục 6.
UndefinedMetricWarning, precision bị đặt bằng 0	Ở ngưỡng đang xét, mô hình không gắn nhãn dương cho bạn nào nên mẫu số bằng 0. Truyền <code>zero_division=1</code> hoặc <code>zero_division=0</code> để nói rõ muốn xử lý ra sao.

Xem tiếp ở trang sau

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
Kết quả khác tài liệu	Quên đặt <code>random_state=17</code> hoặc quên <code>stratify=y</code> trong <code>train_test_split</code> . Hai tham số này phải giữ nguyên thì cách chia mới giống.

Một lời khuyên chung

Lỗi nguy hiểm nhất của bài này không phải lỗi làm chương trình dừng, mà là lỗi làm chương trình chạy trơn tru nhưng cho kết quả vô nghĩa. Ví dụ viết sai dấu trong hàm sigmoid, hoặc đọc nhầm precision thành recall. Cách phòng tránh tốt nhất là luôn tự kiểm chứng bằng một con số đã biết trước, chẳng hạn $\sigma(0)$ phải bằng 0.5.